ONE-DAY REVISION SHEET · VIVA QUESTIONS · ANSWER-WRITING STRATEGY

PART 1 — ⏱️ ONE-DAY REVISION SHEET (1–2 hours)

1. Key definitions (one line each)

- **Distributed system:** networked components that communicate & coordinate **only by messages**.
- **Middleware:** layer masking heterogeneity (RMI, RPC, CORBA).
- **Marshalling:** data → byte stream. **Stub/Proxy:** client-side representative. **Skeleton/Dispatcher:** server-side.
- **RMI:** call method of remote object. **RPC:** call remote procedure.
- **Happened-before →:** same process order + send→receive + transitive.
- **Global state:** process states + channel states. **Consistent cut:** no orphan messages.
- **Mutual exclusion:** one process in CS at a time. **Election:** choose one coordinator (highest id).
- **Multicast:** one → group. **Consensus:** all agree on one proposed value.

2. Characteristics (Unit-I)

Concurrency · **No global clock** · **Independent failures** (core 3) + heterogeneity, openness, scalability, transparency, fault tolerance, security, resource sharing.

3. Client–Server vs P2P vs Grid vs Cloud

	C-S	P2P	Grid	Cloud
Idea	server serves clients	all equal	many orgs share compute	rent services
Control	central	none	virtual org/scheduler	provider
SPOF	server	no	scheduler	no (redundant)
Eg	Web	BitTorrent	SETI@home, CERN	AWS, Azure
Billing	–	free	project	pay-per-use

4. System models

- **Architectural:** layers+middleware, client-server, multiple servers, proxy/cache, P2P, mobile code, mobile agents, thin client.
- **Fundamental: Interaction** (latency, bandwidth, jitter, drift; sync/async) · **Failure** (omission, arbitrary, timing) · **Security** (threats, crypto, authentication).

5. Message passing vs Shared memory

MP	SM
send/receive, works across network	common memory, same machine (DSM emulation)
slower, scalable, implicit sync	faster, needs locks, poor scalability

6. RMI architecture (draw this!)

```
Client → PROXY(stub) → comm+remote ref module ═net═ comm module → DISPATCHER → SKELETON → SERVANT
```

Semantics: maybe / at-least-once / **at-most-once (Java RMI)**.

7. Java RMI (recall)

Interface `extends Remote` + `throws RemoteException` → Impl `extends UnicastRemoteObject` → Server: `createRegistry(1099)`, `rebind("name", obj)` → Client: `getRegistry`, `lookup("name")`, call.

8. RPC vs RMI

RPC	RMI
procedures, C-like, IDL	objects/methods, Java
no object passing	objects & remote references
client stub / server stub	stub / skeleton, GC, registry

9. Events & notifications

Publisher → **Event service** → Subscribers; asynchronous, decoupled (topic/content based).

10. Lamport clock — RULES

```
event:   L = L + 1
send:    L = L + 1; attach L
receive: L = max(L, t) + 1
a → b  ⇒  L(a) < L(b)   (converse FALSE)
```

Vector clock: V[i]++; on receive V=max(V,t) then V[i]++. a→b ⇔ V(a)<V(b).

11. Physical clocks

- **Skew** = difference now. **Drift** = rate ($\rho \approx 10^{-6} \Rightarrow 86.4$ ms/day). Resync interval $\leq \delta/2\rho$.
- **Cristian:** new = Ts + RTT/2 (± (RTT/2 − min)). **Berkeley:** master averages, sends adjustments. **NTP:** strata; $\theta = ((t1-t0)+(t2-t3))/2$.

12. Logical vs Physical

Logical: order, no sync, no drift, counters. Physical: real time, needs sync, affected by drift/skew.

13. Global state / snapshot (Chandy–Lamport)

Initiator: record state → send MARKER on all outgoing → record incoming. Receive first MARKER: record state, that channel = ∅, send markers, record other incoming. Later marker: channel = messages received since recording. FIFO channels. Example 450+350+180+20 = 1000.

14. Distributed debugging

Monitor + vector timestamps → consistent global states → check predicate. **Possibly Φ** (some path) / **Definitely Φ** (all paths). Stable vs non-stable.

15. Mutual exclusion

ME1 safety · ME2 liveness · ME3 ordering. | Alg | Msgs | |---|---| | Central | 3 | | Ring | 1..N−1 | | **Ricart–Agrawala** | **2(N−1)** | | Lamport | 3(N−1) | | Maekawa | ~3√N | RA: WANTED → multicast REQUEST(T,i) → wait N−1 replies → HELD. On request: if HELD or (WANTED & my ts smaller) **queue**, else **reply**. Exit → reply queued.

16. Election

- **Bully:** ELECTION → higher ids; OK; highest alive → COORDINATOR. Sync system, all IDs known. **N−2 best, O(N²) worst.**
- **Ring:** ELECTION(id) circulates, larger id substituted; own id returns → winner; ELECTED circulates. Async OK. **2N best, 3N−1 worst.**

17. Multicast

Reliable: integrity, validity, agreement. Ordering: FIFO (seq numbers), causal (vector clocks), total (sequencer/ISIS).

18. Consensus

Termination · Agreement · Validity(Integrity). Crash: f+1 rounds. Byzantine: N ≥ 3f+1. **FLP:** async + 1 crash ⇒ no guaranteed deterministic consensus. Related: Byzantine generals, interactive consistency, total-order multicast.

19. Keywords to sprinkle in answers

transparency · middleware · marshalling · proxy · dispatcher · at-most-once · latency · jitter · omission/Byzantine · happened-before · concurrent · causality · orphan message · marker · consistent cut · safety/liveness · timestamp · token · coordinator · quorum · agreement/validity/termination.

20. Last-hour checklist

☐ Draw RMI diagram ☐ Write Lamport rules & one example ☐ Write RA pseudo-code ☐ Draw Bully 5-process example ☐ Draw Chandy–Lamport with P1,P2,P3 ☐ Recite consensus 3

conditions ☐ Recite 4-way comparison ☐ Recite MP vs SM ☐ Recite Cristian & Berkeley formulas.

PART 2 — 🎤 30 VIVA / ORAL QUESTIONS

Unit-I 1. **What is a distributed system?** Independent computers connected by a network that coordinate only through messages and appear to the user as one system. 2. **Why do we need distributed systems?** For resource sharing, scalability, reliability and performance which one computer cannot give alone. 3. **What is transparency?** Hiding the distribution — user doesn't know where resources are, how many copies exist or if failures happened. 4. **Difference between client–server and P2P?** In client–server roles are fixed and server is central; in P2P every node is both client and server. 5. **How is grid different from cloud?** Grid pools computers of many organisations for large computations (batch, sharing); cloud rents on-demand services from a provider with pay-per-use. 6. **What is middleware?** Software between OS and application that hides differences of hardware/OS/network (e.g., RMI, CORBA). 7. **Why do we need system models?** They state assumptions clearly so we can design and prove algorithms without depending on a particular system. 8. **What is the difference between architectural and fundamental models?** Architectural = structure/placement of components; fundamental = abstract properties (interaction, failure, security). 9. **What is the difference between synchronous and asynchronous systems?** Synchronous has known bounds on execution/message delay/drift; asynchronous has none, so timeouts can't reliably detect crashes. 10. **What is an internetwork?** Network of networks connected by routers (the Internet). 11. **TCP vs UDP?** TCP is reliable, ordered, connection-oriented; UDP is connectionless, faster but unreliable. 12. **What is IPC and its two types?** Way for processes to exchange data: message passing and shared memory. 13. **Why does DS prefer message passing?** No common physical memory between machines; messages work over networks and give isolation. 14. **What is a remote object?** An object whose methods can be invoked from another process through a remote reference. 15. **How does RMI work?** Client calls the stub, which marshals the call and sends it; server skeleton/dispatcher unmarshals, calls the real object and sends result back. 16. **What is the difference between RPC and RMI?** RPC calls procedures (procedural, no objects); RMI calls methods on remote objects and can pass objects/references. 17. **What is an RMI registry?** A naming service (port 1099) where servers bind names to stubs and clients look them up. 18. **Why does every remote method throw `RemoteException`?** Because network or server failure can occur and the caller must handle it. 19. **What are events and notifications?** Event = something of interest; notification = message telling subscribers about it via an event service (publish–subscribe).

Unit-II 20. **Why do we need synchronization?** No global clock and no shared memory, so processes need methods to agree on order, resource access, leader and decisions. 21. **What is the happened-before relation?** Causal ordering: same-process order, send→receive, and transitivity; events without a path are concurrent. 22. **How does a Lamport clock work?** Increment before every event; send timestamp with message; on receive set clock = max(local, message ts)+1. 23. **Can Lamport clocks detect concurrency?** No; vector clocks can. 24. **Difference between clock skew and drift?** Skew is the difference between two clocks now; drift is the rate at which a clock deviates from real time. 25. **How does Cristian's algorithm set the time?** Time = server time + RTT/2. 26. **What is a global state /**

consistent cut? Local states of all processes + messages in transit; consistent if no message is received without being sent inside the cut. 27. **How does the snapshot algorithm work?** Initiator records its state and sends markers; others record on the first marker; messages arriving between recording and marker form channel state. 28. **State the requirements for mutual exclusion.** Safety (one in CS), liveness (no deadlock/starvation), ordering (happened-before fairness). 29. **How does the Bully algorithm elect a leader?** A process that detects failure sends ELECTION to higher IDs; if none answers it becomes coordinator; otherwise the higher process continues; highest alive ID wins. 30. **What is consensus and its conditions?** All correct processes decide the same value; requires termination, agreement and validity.

Bonus quick ones: Which ring election message count is worst? 3N−1. Ricart–Agrawala messages? 2(N−1). Byzantine tolerance bound? N ≥ 3f+1. Rounds for f crashes? f+1.

PART 3 — ✍️ HOW TO WRITE 5-MARK AND 10-MARK ANSWERS

3.1 The universal skeleton

Step	What to write	Time
1	**Definition** (1–2 lines, boxed/underlined keyword)	1 min
2	**Concept in simple words** (2–3 lines)	1 min
3	**Diagram** (neat, labelled, numbered arrows, caption)	2–3 min
4	**Working / steps** (numbered points; algorithm/pseudocode if asked)	3–5 min
5	**Example** (small numeric or real-life)	2 min
6	**Advantages / disadvantages** (3+3 short points)	1 min
7	**Conclusion** (1 line linking to distributed systems)	30 s

5-mark answer = ~1 page: definition (1) + diagram (1.5) + working (1.5) + example/adv-disadv (1). **10-mark answer** = ~3 pages: definition + explanation + 2 diagrams + detailed steps + example + comparison table + adv/disadv + conclusion.

Tips 1. Underline keywords (transparency, marshalling, happened-before…). 2. Always draw diagrams with a ruler/pencil and **label arrows with message names**. 3. Use tables for comparisons (minimum 6 rows). 4. For algorithms: purpose → assumptions → steps → example → messages → pros/cons. 5. For numericals: **show the rule, then a table of calculations, then the final boxed answer**. 6. Write "Ans:" and box the final result. 7. Don't leave any question blank – attempt definition + diagram at least. 8. For Java code, write the 4 files with brief comments and the sample output.

3.2 SAMPLE 5-MARK ANSWER

Q. Explain Lamport's logical clock with rules and an example. [5]

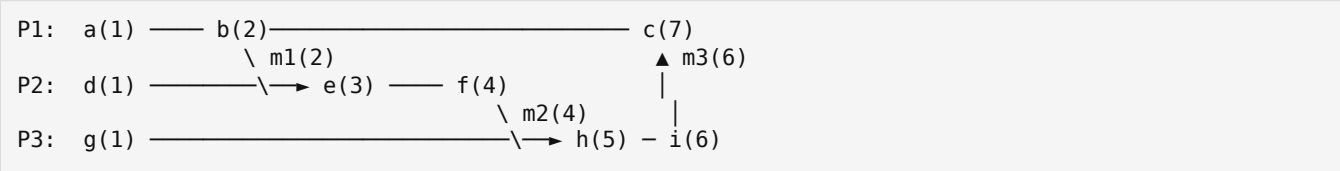
Definition: A *Lamport logical clock* is a software counter kept at each process that assigns timestamps to events such that **if a → b then L(a) < L(b)**. It orders events by causality without using physical clocks.

Why needed: In a distributed system there is **no global clock** and physical clocks drift, so events on different computers cannot be ordered by real time.

Happened-before (→): (1) events in same process ordered; (2) send(m) → receive(m); (3) transitivity. Events not related are **concurrent**.

Rules - LC1: Before each event, `L = L + 1`. - LC2: When sending, attach current `L` to the message. - LC3: On receiving (m, t): `L = max(L, t) + 1`.

Diagram (timeline)



Working: e receives m1 (t = 2) so L = max(1, 2)+1 = 3; h receives m2 (t = 4): max(1,4)+1 = 5; c receives m3 (t = 6): max(2,6)+1 = 7.

Advantages: simple (one integer); preserves causality; no clock sync. **Limitations:** if L(a) < L(b) we cannot say a → b (a and b may be concurrent, e.g., a(1) and d(1)); cannot detect concurrency – vector clocks are needed.

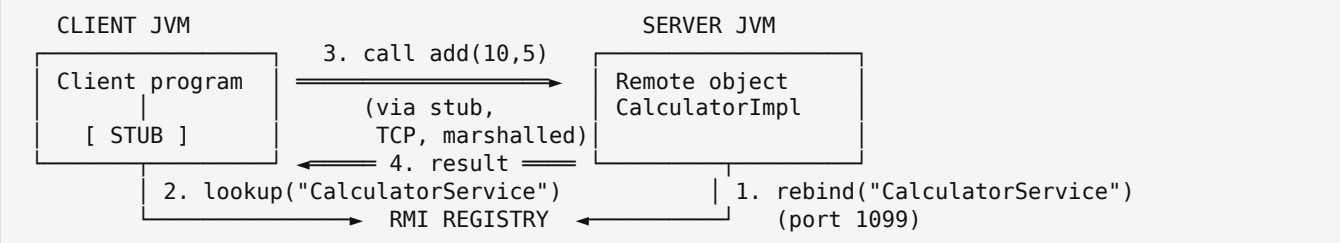
Conclusion: Lamport clocks give a consistent ordering of events in distributed systems and form the basis for total ordering in algorithms like Ricart–Agrawala mutual exclusion.

3.3 SAMPLE 10-MARK ANSWER

Q. Design a simple Java RMI application (Calculator). Explain architecture, remote interface, server, client, registry and execution steps with code and output. [10]

1. Introduction (1 mark). *Java RMI (Remote Method Invocation)* lets an object in one JVM invoke methods of an object in another JVM (possibly on another machine) as if it were local. It follows the distributed-object model, hides communication through **stubs**, and gives **at-most-once** semantics.

2. Architecture diagram (2 marks)



3. Components (2 marks) | Component | Role in the calculator | |---|---| | Remote interface `Calculator` | extends `Remote`; declares add/subtract/multiply/divide with `throws RemoteException`

| | Remote object `CalculatorImpl` | implements the interface, extends `UnicastRemoteObject` (exports the object) | | Server | creates the object, starts the registry, `rebind`s the name | | Registry | naming service; maps "CalculatorService" → stub | | Stub | client-side proxy that marshals the call | | Client | looks up the stub and invokes methods |

4. Code (3 marks)

```
// Calculator.java
import java.rmi.*;
public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
}
// CalculatorImpl.java
import java.rmi.*; import java.rmi.server.*;
public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    public CalculatorImpl() throws RemoteException { super(); }
    public int add(int a, int b) { return a + b; }
    public int multiply(int a, int b) { return a * b; }
}
// CalculatorServer.java
import java.rmi.registry.*;
public class CalculatorServer {
    public static void main(String[] args) throws Exception {
        Registry r = LocateRegistry.createRegistry(1099);
        r.rebind("CalculatorService", new CalculatorImpl());
        System.out.println("Server ready");
    }
}
// CalculatorClient.java
import java.rmi.registry.*;
public class CalculatorClient {
    public static void main(String[] args) throws Exception {
        Registry r = LocateRegistry.getRegistry("localhost", 1099);
        Calculator c = (Calculator) r.lookup("CalculatorService");
        System.out.println("10 + 5 = " + c.add(10, 5));
        System.out.println("10 * 5 = " + c.multiply(10, 5));
    }
}
```

5. Execution steps (1.5 marks) 1. `javac *.java` 2. `java CalculatorServer` (registry starts, object registered) 3. `java CalculatorClient` 4. Client contacts registry → gets stub → calls `add(10,5)` → stub marshals → server executes → result unmarshalled → printed.

6. Sample output (0.5 mark)

```
Server ready                    (server terminal)
10 + 5 = 15                     (client terminal)
10 * 5 = 50
```

Advantages: simple, OO, automatic stubs/GC, object passing. **Limitations:** Java-only, slower than local calls, firewall issues. **Conclusion:** Java RMI implements the distributed-object model by hiding network communication behind proxies, making distributed programming close to normal object-oriented programming.

3.4 Common mistakes to avoid

Mistake	Fix
Writing Unit-III/IV (HDFS, IaaS) in Unit-I/II answers	Stay inside Unit-I/II; mention cloud only as an *example*
Saying L(a)<L(b) ⇒ a→b	Only a→b ⇒ L(a)<L(b)
Confusing skew and drift	Skew = difference; drift = rate
Bully "lowest id wins"	**Highest** id wins in both Bully and Ring
Forgetting `throws RemoteException`	Always include; interface must extend `Remote`
Diagrams without labels	Label every arrow with message name/step number
Ignoring assumptions of algorithms	FIFO for snapshot; synchronous for Bully; reliable delivery for RA
Writing only theory for numericals	Show rule → table → boxed answer

---

You now have: notes (01, 02) → priorities (03) → questions (04, 05) → timed papers (06) → last-day sheet (this file). **Good luck for the mid-sem!**